



ESCUELA DE
POSTGRADO

Maestría en Automatización Industrial · UTH 2026.4

Programación · Proyecto Final

Agente de Mantenimiento Industrial con IA

Guía del estudiante y rubrica de evaluación

Valor: 40 puntos (Proyecto 30 + Presentación 10)

Cátedra: PhD(c) Luis Loo · 2026

En una frase

Vas a construir, de forma individual, un agente de IA que asiste con una tarea real de mantenimiento industrial: recibe datos, razona, consulta documentación técnica, ejecuta herramientas y avisa a un humano. El cerebro del agente es Claude Code; tú pones las herramientas y el conocimiento.

Lo esencial

Que entregas: el repositorio de tu agente (en GitHub) y una demo en vivo de 5 a 7 minutos.

Cuanto vale: 40 puntos. Proyecto 30, presentación 10.

Cuando: demo el sábado de la Semana 5 (S5), junio 2026.

Con que cerebro: Claude Code. No necesitas API key de OpenAI ni Gemini, ni pagar por tokens.

1. De que trata el proyecto

Un agente no es solo un chatbot. Es un sistema que percibe una entrada, razona con un modelo de lenguaje, decide y actúa usando herramientas, y comunica el resultado. Tu agente debe cubrir estas cinco capacidades:

- **Recibir.** Una entrada: lecturas de sensores, un texto del operador o un evento de alarma.
- **Razonar.** Un LLM con un rol específico de mantenimiento (tu system prompt).
- **Consultar.** Documentación técnica via RAG (manuales, normas, fichas).
- **Ejecutar.** Al menos una herramienta: analizar datos, consultar un SCADA simulado, crear una orden de trabajo, etc.
- **Notificar.** El resultado llega a un humano por un canal real: correo, Telegram, WhatsApp o dashboard.

El proyecto es individual. Cada estudiante elige una variante (sección 3) y entrega su propio agente. Está bien que se ayuden entre ustedes, pero el código y la demo son tuyos.

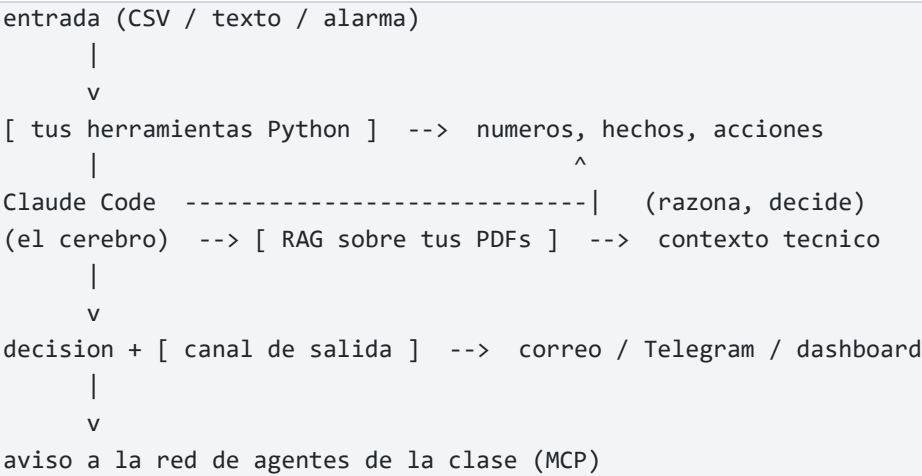
2. El cerebro de tu agente es Claude Code

En este curso el motor de razonamiento de tu agente es Claude Code, la misma herramienta que ya instalaste y usaste en clase. No vas a llamar a una API de OpenAI ni de Gemini. La ventaja: cero API key, cero costo por token y menos cosas que se pueden romper el día de la demo.

El reparto de trabajo es claro:

- **Tú escribes herramientas en Python.** Scripts deterministas que hacen calculo o accion: leen un CSV, evaluan una norma, mandan un correo. Nada de IA dentro de ellos, solo codigo.
- **Claude Code razona.** Lee tu archivo CLAUDE.md (su system prompt), ejecuta tus herramientas, interpreta los números que devuelven, consulta el RAG y emite la decisión.
- **Tú lo invocas.** Con un comando propio (por ejemplo /diagnosticar) o pidiéndoselo en lenguaje natural dentro de la sesión.

El flujo, de principio a fin:



Referencia

Tienes un ejemplo de oro: MotorVigia, el agente de demostración que vimos en clase (variante 1, diagnóstico de bombas y motores). Estúdialo para ver cómo se ven en la práctica el CLAUDE.md, las herramientas, el RAG y el canal de correo. La sección 6 explica cómo aprovecharlo sin copiarlo.

3. Elige tu variante

Escoge una. Si quieres otra, puedes seleccionarla.

#	Variante	Qué construye
1	Diagnóstico de bombas / motores	Le pasas lecturas de sensores y te dice qué tiene el equipo y qué hacer.

2	Mantenimiento predictivo	Con datos sintéticos históricos, el agente anticipa una falla antes de que ocurra.
3	Generador de órdenes de trabajo	De un texto libre del operador a una orden de trabajo estructurada.
4	Asistente de seguridad industrial	RAG sobre normativa (OSHA / STSS) que responde dudas de procedimiento.
5	Otra	Tu idea, cubriendo los elementos del agente MotorVigia.

4. Componentes obligatorios del agente

Tu agente debe tener estos seis componentes más un repositorio bien armado. Cada uno se evalúa (sección 8). Abajo, que se espera de cada uno y donde verlo en MotorVigia.

4.1 Clase(s) POO propia(s)

Al menos una clase tuya que modele tu dominio (por ejemplo Equipo, Lectura, OrdenTrabajo, Diagnostico). Con atributos, métodos y, si encaja, herencia. Es el milestone de POO de la Tarea 4.

En MotorVigia: src/modelos.py define Lectura, Motor y Diagnostico.

4.2 RAG sobre documentación técnica

Indexa 1 o 2 documentos técnicos (manual, ficha, norma) y recupera los fragmentos relevantes para fundamentar la respuesta. El agente debe citar de donde saco la información, no inventarla.

En MotorVigia: src/rag.py (TF-IDF) sobre data/conocimiento/ y un PDF de la norma ISO 10816.

4.3 Herramienta(s) vía tool calling

Al menos una herramienta: un script de Python que el agente ejecuta para obtener datos o actuar (leer un log, evaluar una norma, abrir una orden, mandar un mensaje). Dos o tres se ve mejor.

En MotorVigia: tools/analizar.py, consultar_norma.py, simular_falla.py, notificar.py.

4.4 Canal de salida

La decisión del agente tiene que llegar a un humano por un canal real: correo (smtplib o un servicio como Brevo o Resend), Telegram, WhatsApp o un dashboard (Streamlit). No basta con imprimir en consola.

En MotorVigia: src/notificaciones.py envía un correo HTML con el diagnóstico.

4.5 System prompt documentado (CLAUDE.md)

El rol, los criterios y el flujo de tu agente, escritos y específicos de tu variante: quién es, cómo razona, qué reglas sigue, cuándo usa cada herramienta. Va en el archivo CLAUDE.md, que Claude Code lee automáticamente al abrir el repo.

En MotorVigia: CLAUDE.md y prompts/system_agente.md.

4.6 Conexión a la red de agentes de la clase (MCP) [OBLIGATORIO]

Tu agente debe conectarse al servidor MCP de la clase (uthAgentes) y, como mínimo, registrarse y enviar o leer un mensaje. Es cómo tu agente le avisa al resto de la clase que pasó algo (por ejemplo, que detectó una alarma) o consulta el estado de otros. Las credenciales del servidor te las da el instructor y van en tu archivo .env.

En MotorVigia: `tools/avisar_mcp.py` (difunde un aviso) y `tools/leer_mcp.py` (lee su bandeja).

4.7 Repositorio e ingeniería

Repo en GitHub con README claro, estructura ordenada, requirements.txt, un .env.example (sin valores reales) y un .gitignore que deje el .env fuera. Cero secretos en el código. Commits que cuenten la historia del trabajo.

5. Cómo proceder: ruta paso a paso

Una secuencia realista para llegar a la demo sin trasnochar el viernes:

- Paso 1.** Crea el repo y abre Claude Code dentro de la carpeta. Pídele que lea PROYECTO.md y te resuma tu siguiente paso.
- Paso 2.** Diseña tu primera clase POO (`src/modelos.py`) y un dato de entrada simulado (un CSV o un JSON pequeño).
- Paso 3.** Escribe tu primera herramienta (un script que lea ese dato y devuelva algo útil en JSON).
- Paso 4.** Reúne 1 o 2 documentos técnicos y arma el RAG. Pruébalo con una consulta.
- Paso 5.** Escribe tu CLAUDE.md: rol del agente, reglas y el flujo que debe seguir.
- Paso 6.** Conecta el canal de salida (correo es lo más simple) y Pruébalo en modo simulado primero.
- Paso 7.** Conecta la red MCP de la clase: regístrate y manda un mensaje de prueba.
- Paso 8.** Corre el agente de punta a punta. Ajusta el CLAUDE.md hasta que razone bien.
- Paso 9.** Ensaya la demo cronometrada y deja el repo limpio.

Milestones y fechas:

Cuando	Hito	Que debe estar listo
Ya (después de S3)	Repo + primera clase POO	Repo creado, una clase en <code>src/modelos.py</code> , variante elegida.
Antes de S5 viernes	Datos + conocimiento	Entrada simulada (CSV/JSON) y 1 o 2 documentos para el RAG.
S5 viernes	RAG + 1 herramienta	Consultar el RAG y ejecutar una herramienta desde Claude Code.
Entre viernes y sábado	Salida + red MCP	Notificación probada punta a punta y conexión a la red de la clase.
S4 sábado	Demo + repo limpio	Ensayo cronometrado, README, .env fuera del repo, commits.

6. El ejemplo de referencia: MotorVigia

MotorVigia es el agente que vimos en clase. Es la variante 1 (diagnóstico de bombas y motores) y cumple los seis componentes. Úselo como mapa, no como molde.

Qué mirar en el:

- Cómo está escrito el CLAUDE.md: el rol, las reglas y los pasos del flujo.
- Cómo una herramienta recibe argumentos y devuelve JSON limpio que el agente pueda leer.
- Cómo el RAG carga documentos y devuelve fragmentos con su fuente.
- Cómo el canal de correo arma un mensaje y como se usa el modo simulado para probar sin enviar.

Integridad académica

No copies MotorVigia tal cual. Si tu variante es la 1, cambia el enfoque, el dato, el documento del RAG o el canal para que sea tuyo. Si es otra variante, MotorVigia solo te muestra el patrón. En la demo te voy a preguntar por tu agente: tienes que entender cada funcionalidad que entregues, la hayas desarrollado tú o Claude Code.

7. La presentación: demo en vivo (5 a 7 min)

La presentación vale 10 puntos. Lo que más pesa es que el agente corra de verdad frente a la clase, no sólo diapositivas. Guión sugerido:

Min	En pantalla	Que demuestras
0:00 - 1:00	Contexto: que problema de mantenimiento resuelve tu agente.	El porqué.
1:00 - 2:00	Arquitectura en 1 diapositiva: entrada, Claude Code, herramientas, RAG, salida.	Entiendes tu diseño.
2:00 - 4:00	Corres el agente: entra un dato y Claude razona invocando tus herramientas.	Tool calling + razonamiento.
4:30 - 5:30	El RAG cita la documentación que fundamenta la decisión.	RAG real.
5:30 - 6:30	Llega la notificación al canal y el aviso a la red de la clase.	Salida + MCP.
6:30 - 8:00	Cierre: limitaciones y que mejorarías.	Pensamiento crítico.

8. Evaluacion: rúbrica de 40 puntos

8.1 Proyecto (30 puntos)

#	Componente del agente	Que se evalua	Pts
1	Clase(s) POO propia(s)	Al menos una clase propia, bien diseñada (atributos, métodos; idealmente herencia) que modela tu dominio.	4
2	RAG sobre documentación	Indexa 1 ó 2 documentos técnicos y recupera los fragmentos que fundamentan la respuesta.	4
3	Herramienta(s) (tool calling)	Al menos una herramienta que el agente ejecuta para obtener datos o actuar.	4
4	Canal de salida	La decisión del agente llega a un humano por un canal real (correo, Telegram, WhatsApp, dashboard).	4
5	System prompt (CLAUDE.md)	Rol, criterios y flujo del agente, documentados y específicos de tu variante.	4
6	Red de agentes de la clase (MCP)	El agente se registra y envía o lee al menos un mensaje en la red uthAgentes.	4
7	Integración y funcionamiento	Corre de punta a punta (entrada, razonamiento, RAG y herramientas, salida) de forma reproducible y coherente.	4
8	Repo e ingeniería	Estructura clara, README, .env y .gitignore (cero secretos en el código), commits con historia.	2
TOTAL PROYECTO			30

En cada componente: puntaje completo si está implementado y funciona; parcial si está presente pero incompleto o con fallas; cero si está ausente.

8.2 Presentación (10 puntos)

Criterio	Que se evalua	Pts
Demo en vivo funcional	El agente corre de verdad frente a la clase (no solo diapositivas): entra un dato, razona y produce una salida.	4
Dominio técnico	Explicas tu system prompt, tus herramientas y tu RAG; respondes preguntas sobre tu código.	3
Claridad y estructura	Contexto del problema, arquitectura del agente y cierre, en orden y entendibles.	2

Tiempo y profesionalismo	Te ajustas a 5 a 7 minutos, con una presentación ordenada.	1
TOTAL PRESENTACIÓN		10
Total del proyecto final: 40 puntos = Proyecto 30 + Presentación 10.		

9. Reglas y buenas prácticas

- **Secretos en .env, nunca en el código.** Contraseñas, tokens y el del servidor MCP van en .env, que queda fuera del repo (.gitignore). Sube solo un .env.example sin valores.
- **Repo público y limpio.** README que explique que hace tu agente y como correrlo. Borra archivos basura antes de entregar.
- **Commits frecuentes.** Cada vez que algo funciona, haz commit. Si algo se rompe, vuelves atrás sin perder.
- **Usa Claude Code como copiloto, no como piloto.** Pídele cosas concretas (una clase, una herramienta), lee lo que escribe y entiéndelo. En la demo respondes tu.
- **Pide ayuda temprano.** Si te trabas más de 15 minutos, pregunta en el foro de Canvas o agenda conmigo. Un proyecto que muere en silencio es el único que de verdad falla.

10. Checklist de entrega

Antes del sábado de S5, confirma que tienes todo:

- ☐ Repo en GitHub, publico, con README claro.
- ☐ Al menos una clase POO propia.
- ☐ RAG sobre 1 o 2 documentos tecnicos, funcionando.
- ☐ Al menos una herramienta que el agente ejecuta.
- ☐ Canal de salida real, probado punta a punta.
- ☐ CLAUDE.md con el rol y el flujo del agente.
- ☐ Conexión a la red MCP de la clase, con un mensaje de prueba enviado o leído.
- ☐ El agente corre de principio a fin sin errores fatales.
- ☐ .env fuera del repo; .env.example incluido; cero secretos en el codigo.
- ☐ Demo de 6 a 8 minutos ensayada y cronometrada, con plan B de respaldo.

11. Recursos y si te trabas

- Material del curso S1 a S4 en Canvas (notebooks, decks, Tarea 4 de POO).
- El repo de MotorVigia como ejemplo de referencia.
- Documentación de Claude Code: docs.claude.com/claude-code.